EXPERIMENT 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

Introduction to Blockchain

Blockchain is a decentralized and distributed digital ledger that records transactions in a secure and transparent manner. Instead of relying on a central authority, multiple nodes in a network maintain copies of the same ledger. Data is stored in blocks that are linked together sequentially using cryptographic hash values.

Structure of a Block

Each block in the blockchain contains an index, timestamp, list of transactions, proof value, and the hash of the previous block. The first block created in the chain is called the Genesis Block. The previous hash field connects one block to another, forming a continuous chain.

Hashing

The SHA-256 hashing algorithm is used to generate a unique hash value for each block. Hashing ensures data integrity because even a small change in the block data produces a completely different hash. This makes the blockchain tamper-resistant and secure.

Proof of Work

Proof of Work is the consensus mechanism used to validate new blocks. It requires solving a computational puzzle before adding a block to the chain. A block is considered valid only when the generated hash satisfies a specific condition. This prevents unauthorized block creation and secures the network.

Mining

Mining is the process of solving the Proof of Work and adding a new block to the blockchain. During mining, pending transactions are verified and included in the new block. Once the proof is found, the block is appended to the chain.

Decentralized Network and Synchronization

Multiple blockchain nodes are created by running the application on different port numbers. Each node maintains its own copy of the blockchain. Nodes are connected to form a network, and the Longest Chain Rule is applied to maintain consistency. If a node finds a longer valid chain, it replaces its current chain to ensure synchronization across the network.

```
# Module 2 - Create a Cryptocurrency

# To be installed:
# Flask==0.12.2: pip install Flask==0.12.2
# Postman HTTP Client: https://www.getpostman.com/
# requests==2.18.4: pip install requests==2.18.4
```

```
# Importing the libraries
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4                                  # Generate a unique id that is in hex
from urllib.parse import urlparse                       # To parse url of the nodes

# Part 1 - Building a Blockchain

class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []                          # Adding transactions before they are added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set()                              # Set is used as there is no order to be maintained as the
nodes can be from all around the globe

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': self.transactions}     # Adding transactions to make the blockchain a
cryptocurrency
        self.transactions = []                          # The list of transacction should become empty after they
are added to a block
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()
```

```
    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
        return True

    # This method will add the trqnsaction to the list of trnsactions
    def add_transaction(self, sender, receiver, amount):
        self.transactions.append({'sender': sender,
                                  'receiver': receiver,
                                  'amount': amount})
        previous_block = self.get_previous_block()
        return previous_block['index'] + 1                          # It will return the block index to which the
transaction should be added

    # This function will add the node containing an address to the set of nodes created in init function
    def add_node(self, address):
        parsed_url = urlparse(address)                              # urlparse will parse the url from the address
        self.nodes.add(parsed_url.netloc)                           # Add is used and not append as it's a set. Netloc will
only return '127.0.0.1:5000'

    # Consensus Protocol. This function will replace all the shorter chain with the longer chain in all the nodes on the
network
    def replace_chain(self):
        network = self.nodes                                        # network variable is the set of nodes all around the globe
        longest_chain = None                                        # It will hold the longest chain when we scan the
network
        max_length = len(self.chain)                                # This will hold the length of the chain held by the
node that runs this function
        for node in network:
            response = requests.get(f'http://{node}/get_chain')     # Use get chain method already created to  get
the length of the chain
            if response.status_code == 200:
                length = response.json()['length']                  # Extract the length of the chain from get_chain
fiunction
                chain = response.json()['chain']
                if length > max_length and self.is_chain_valid(chain):      # We check if the length is bigger and if the
chain is valid then
```

```
            max_length = length                         # We update the max length
            longest_chain = chain                       # We update the longest chain
    if longest_chain:                                 # If longest_chain is not none that means it was replaced
        self.chain = longest_chain                        # Replace the chain of the current node with the longest
chain
        return True
    return False                                      # Return false if current chain is the longest one

# Part 2 - Mining our Blockchain
app = Flask(__name__)
# Creating an address for the node on Port 5000. We will create some other nodes as well on different ports
node_address = str(uuid4()).replace('-', '')                    #
# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount = 1) # Hadcoins to mine the
block (A Reward). So the node gives 1 hadcoin to Abcde for mining the block
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash'],
                'transactions': block['transactions']}
    return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return jsonify(response), 200
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200
@app.route('/add_transaction', methods = ['POST'])                    # Post method as we have to pass something
to get something in return
```

```
def add_transaction():
    json = request.get_json()                                   # This will get the json file from postman. In Postman we
will create a json file in which we will pass the values for the keys in the json file
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json for key in transaction_keys):                # Checking if all keys are available in json
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
    response = {'message': f'This transaction will be added to Block {index}'}
    return jsonify(response), 201                                # Code 201 for creation

# Part 3 - Decentralizing our Blockchain
# Connecting new nodes
@app.route('/connect_node', methods = ['POST'])                         # POST request to register the new nodes
from the json file
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes')                                   # Get the nodes from json file
    if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains the following
nodes:',
                'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201
@app.route('/replace_chain', methods = ['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the longest one.',
                    'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
                    'actual_chain': blockchain.chain}
    return jsonify(response), 200
app.run(host = '0.0.0.0', port = 5001)
```

1. Using Postman to connect all nodes using the /connect_node API

My Collection / **Get data**

Save | Share

POST http://127.0.0.1:5000/connect_node Send

Docs | Params | Authorization | Headers (9) | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
{
  "nodes": ["http://127.0.0.1:5001"]
}
```

Body | Cookies | Headers (5) | Test Results (0/1) | 201 CREATED | 21 ms | 233 B | Save Response

JSON | Preview | Visualize

```
{
    "message": "Nodes connected",
    "total_nodes": [
        "127.0.0.1:5001"
    ]
}
```

2. Add transactions using the /add_transaction API.

My Collection / **Get data**

Save | Share

POST http://127.0.0.1:5001/add_transaction Send

Params | Authorization | Headers (8) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Beautify

```
{
    "sender": "Jatin",
    "receiver": "Jatin2",
    "amount": 50
}
```

Body | Cookies | Headers (5) | Test Results (0/1) | 201 CREATED | 12 ms | 226 B | Save Response

JSON | Preview | Visualize

```
{
    "message": "This transaction will be added to Block 2"
}
```

POST Get data

No environment

My Collection / Get data

Save Share

POST http://127.0.0.1:5000/add_transaction Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "sender": "Nayaab",
  "receiver": "Komal",
  "amount": 500
}
```

Body Cookies Headers (5) Test Results (0/1) 201 CREATED 5 ms 213 B Save Response

{} JSON Preview Visualize

```
{
    "message": "Transaction added to block 5"
}
```

POST Get data

No environment

My Collection / Get data

Save Share

POST http://127.0.0.1:5000/add_transaction Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "sender": "Veydant",
  "receiver": "Bhumisha",
  "amount": 100
}
```

Body Cookies Headers (5) Test Results (0/1) 201 CREATED 6 ms 213 B Save Response

{} JSON Preview Visualize

```
{
    "message": "Transaction added to block 5"
}
```

3. Performed mining using the /mine_block API.

GET Get data

No environment

My Collection / Get data

Save Share

GET http://127.0.0.1:5000/mine_block Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

Body Cookies Headers (5) Test Results (1/1) 200 OK 29 ms 439 B Save Response

JSON Preview Visualize


```
{
    "block": {
        "index": 4,
        "previous_hash": "92fe4b254aeeef8d3af22fdb1fd8f371c54ca0c638cfcf73e9321f7056ff3286",
        "proof": 21391,
        "timestamp": "2026-02-09 13:14:18.311451",
        "transactions": [
            {
                "amount": 1,
                "receiver": "1abb1da37cc949fabcae474a656bfbb3",
                "sender": "0"
            }
        ]
    },
    "message": "Block mined"
}
```


Activate Windows
Go to Settings to activate Windows.

4. View the blockchain using the /get_chain API.

My Collection / Get data

Save Share

GET http://127.0.0.1:5000/get_chain Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

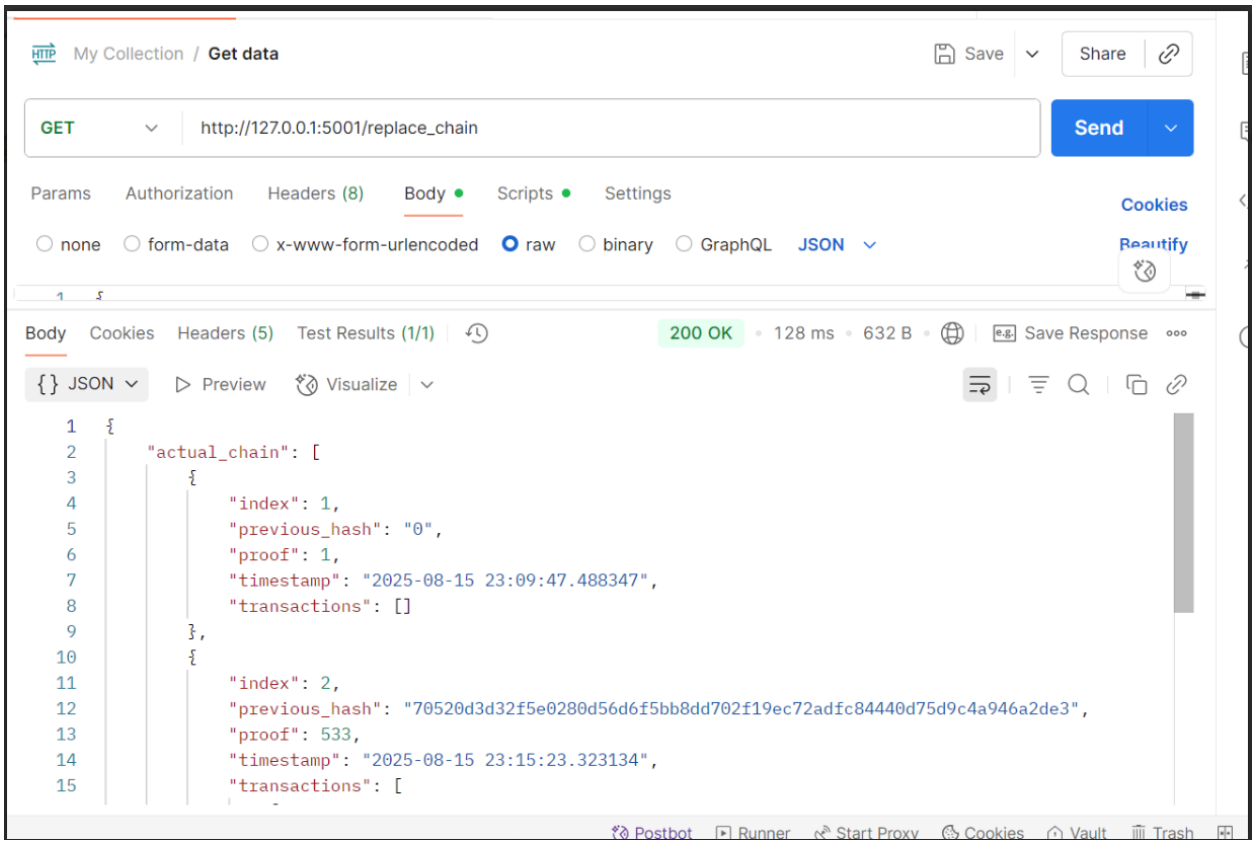
Body Cookies Headers (5) Test Results (1/1) 200 OK 5 ms 1005 B Save Response

JSON Preview Visualize

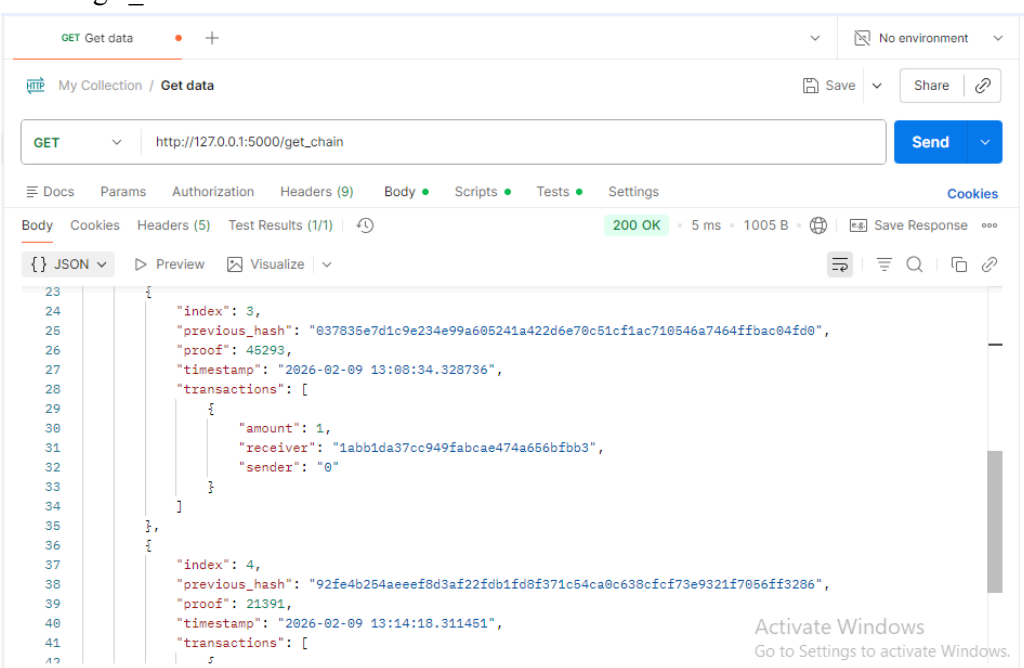

```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-02-09 13:03:46.168897",
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "4160062f27b9ad9bb61630fe8987a85d5080b5b31b387ce175c1066b83495fd1",
            "proof": 533,
            "timestamp": "2026-02-09 13:06:29.797023",
            "transactions": [
                {
                    "amount": 1,
                    "receiver": "1abb1da37cc949fabcae474a656bfbb3",
                    "sender": "0"
```


Activate Windows
Go to Settings to activate Windows.

5. Synchronize blockchain across nodes using the /replace_chain API.


My Collection / Get data
Save
Share
GET
http://127.0.0.1:5001/replace_chain
Send
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
Body
Cookies
Headers (5)
Test Results (1/1)
200 OK
128 ms
632 B
Save Response
JSON
Preview
Visualize
{
"actual_chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2025-08-15 23:09:47.488347",
"transactions": []
},
{
"index": 2,
"previous_hash": "70520d3d32f5e0280d56d6f5bb8dd702f19ec72adfc84440d75d9c4a946a2de3",
"proof": 533,
"timestamp": "2025-08-15 23:15:23.323134",
"transactions": [
Postbot
Runner
Start Proxy
Cookies
Vault
Trash

6. get_chain


GET Get data
No environment
My Collection / Get data
Save
Share
GET
http://127.0.0.1:5000/get_chain
Send
Docs
Params
Authorization
Headers (9)
Body
Scripts
Tests
Settings
Cookies
Body
Cookies
Headers (5)
Test Results (1/1)
200 OK
5 ms
1005 B
Save Response
JSON
Preview
Visualize
{
"index": 3,
"previous_hash": "037835e7d1c9e234e99a605241a422d6e70c51cf1ac710546a7464ffbac04fd0",
"proof": 45293,
"timestamp": "2026-02-09 13:08:34.328736",
"transactions": [
{
"amount": 1,
"receiver": "1abb1da37cc949fabcae474a656bfbb3",
"sender": "0"
}
]
},
{
"index": 4,
"previous_hash": "92fe4b254aeeef8d3af22fdb1fd8f371c54ca0c638cfcf73e9321f7056ff3286",
"proof": 21391,
"timestamp": "2026-02-09 13:14:18.311451",
"transactions": [
Activate Windows
Go to Settings to activate Windows.

Conclusion:
The experiment successfully implemented a basic blockchain and cryptocurrency system using Python. Blocks were mined using Proof of Work, transactions were added, and multiple nodes were synchronized using the longest chain rule. The experiment demonstrated the core concepts of blockchain, mining, and decentralized consensus.